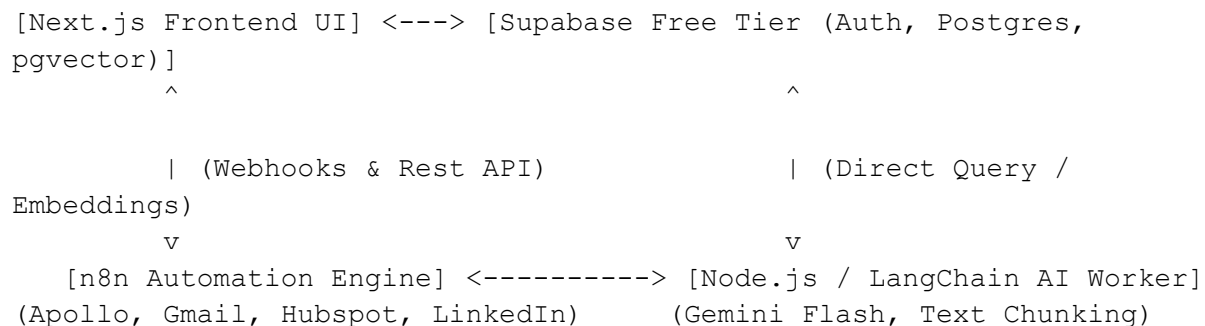


Software Requirements Specification (SRS)

Project: PulseFlow AI (Content-Driven Outreach Automation SaaS)

1. Product Overview & System Architecture

PulseFlow AI is a B2B SaaS platform that allows users to upload content assets (YouTube links, PDFs, blogs), vectorize them, and use automated multi-tool workflows (via n8n and LangChain) to fetch leads from Apollo and generate hyper-personalized cold outreach.



2. Functional Requirements (Features)

Module A: The Semantic Knowledge Base (RAG Engine)

- System Action: User inputs a URL (Blog/YouTube) or uploads a text document.
- Processing: The Node.js app grabs the text, chunks it using LangChain's `RecursiveCharacterTextSplitter`, generates vector embeddings via the HuggingFace Inference API, and stores them in Supabase `pgvector`.
- Value: This builds the "Context Library" used to personalize emails.

Module B: Multi-Tool Integration Engine (n8n Workspace)

- System Action: Instead of writing custom API integration code for every tool, n8n handles the heavy lifting via webhooks.
- Trigger: User clicks "Launch Campaign" on the Next.js frontend.

- n8n Workflow Node 1: Calls the Apollo.io API using the user's targeted filters (e.g., "Founders in SaaS, US").
- n8n Workflow Node 2: Iterates through the fetched leads and sends their data to the LangChain Worker endpoint.
- n8n Workflow Node 3 (Output): Pushes the finalized generated email directly to Gmail (Drafts), Hubspot CRM, or a Slack Channel.

Module C: LangChain Personalization Worker

- System Action: Receives a single lead from n8n + a target objective.
- Step 1: Queries Supabase using cosine similarity to find the top 2 matching content chunks from the Knowledge Base relative to the lead's company industry.
- Step 2: Uses a structured LangChain output prompt (`zod` schema parsing) via Gemini 1.5 Flash to extract lead pain points and generate a custom email.
- Step 3: Returns the generated copy back to n8n to complete the automation sequence.

Module D: The Analytics Dashboard & Landing Page

- Landing Page: Clean modern hero section, "How it works" step-by-step layout showcasing the integrations (n8n, Apollo, Gmail), and pricing cards.
- Dashboard UI: Tabbed navigation tracking:
 - *Knowledge Base*: List of uploaded files/links with processing status badges.
 - *Outreach Matrix*: A live table displaying Apollo data rows, generation status, and preview buttons to see the written emails before they send.

3. Strict 100% Free Production Tech Stack

Component	Technology	Free Tier Boundaries
Frontend UI	Next.js 14+ App Router, Tailwind, Shadcn/ui	Vercel Free Tier (Unlimited deployments, 10s serverless timeout)

Database & Auth	Supabase (PostgreSQL + <code>pgvector</code>)	Supabase Free Tier (500MB DB space, up to 50k monthly active users)
Workflow Engine	n8n Community Edition	Self-Hosted on Render (free web service disk) or Hetzner/Oracle Free Tier
LLM Model	Google Gemini 1.5 Flash via LangChain	Google AI Studio (15 RPM / 1 million TPM completely free)
Embeddings	Hugging Face Transformers API	HF Serverless Architecture (Free for smaller embedding models like <code>all-MiniLM-L6-v2</code>)
Queue Management	Inngest or Upstash Workflow	Upstash Free Tier (10,000 free requests per day, avoids serverless timeouts)

4. Database Schema Design (Supabase PostgreSQL)

`users` (Managed by Supabase Auth automatically)

`knowledge_base` (Stores uploaded context documents)

- `id`: uuid (Primary Key)
- `user_id`: uuid (Foreign Key to `auth.users`)
- `source_url`: text (Optional URL)
- `title`: varchar
- `created_at`: timestamp

`document_chunks` (The vectorized pieces of data)

- `id`: uuid (Primary Key)
- `kb_id`: uuid (Foreign Key to `knowledge_base`, cascade delete)
- `content`: text (The actual parsed text segment)
- `embedding`: vector(384) (Match your HuggingFace model dimensions, build HNSW index)

`leads_campaign` (Stores rows fetched by n8n from Apollo)

- `id`: uuid (Primary Key)
 - `user_id`: uuid (Foreign Key)
 - `first_name`: varchar
 - `last_name`: varchar
 - `email`: varchar
 - `company_name`: varchar
 - `job_title`: varchar
 - `generated_email`: text (Null until LangChain processes it)
 - `status`: varchar ('pending', 'processing', 'completed', 'failed')
-

5. Execution Strategy: Building it "Lightning Fast"

- Step 1 (The UI Core): Spin up a Next.js project using a template. Drop `shadcn Table`, `Card`, and `Button` components onto a single dashboard workspace screen. Connect Supabase Auth so you have a private route.
- Step 2 (The n8n Spine): Boot up n8n. Create your first workflow: An *HTTP Request node* (simulating the UI click) $\rightarrow$ *Apollo Node* (extracts profiles) $\rightarrow$ *Webhook Node* (sends data to your Next.js API).
- Step 3 (The LangChain Muscle): Build a single Next.js API route (`/api/generate-outreach`) that initializes LangChain. Take the incoming n8n lead data, run a Supabase vector RPC search query, fire the prompt to Gemini Flash, and return the payload.

To jump right into development, would you like me to generate the n8n JSON workflow blueprint to map out the tool integrations, or the Supabase SQL migration code to set up the `pgvector` database extension and tables?